

Multi-Module & Optimization

1. What is a multi-module Maven project, and why would you use it?

A: A **multi-module** project is a large project broken down into several smaller, independent Maven projects (modules), all managed by one main parent project. You would use it to **organize** a large application, especially in a microservices architecture, and to enforce clear boundaries between different parts of the system.

2. How do you manage dependencies between modules in a multi-module Maven project?

A: Dependencies between modules are managed by declaring the necessary module in the `<dependencies>` section of the consuming module's `pom.xml`. Crucially, you centralize all version numbers in the `<dependencyManagement>` section of the parent POM. This ensures all modules use consistent versions of shared libraries.

3. How would you optimize a Maven build for a large project with many modules?

A: I would optimize by using the **parallel build** feature (with the `-T` flag) to compile and test independent modules at the same time. I would also focus on keeping the history of each module separate and ensuring tests are fast and efficient.

4. What strategies can you use to speed up slow Maven builds (parallel builds, caching, skipping tests, etc.)?

A: Strategies include enabling **parallel builds** (`-T 4C` or similar) to maximize CPU core use. During rapid development, you can use the `-DskipTests` flag to skip the slow test phase. You should also ensure your internal repository manager is caching dependencies effectively to reduce slow network lookups.

5. Scenario: Your Maven build time keeps increasing over time. What steps would you take to find and fix the root cause?

A: I would investigate dependencies using `mvn dependency:tree` to look for new, unexpectedly large libraries. I would then check the build logs to see if a particular phase, like **integration tests** or static analysis checks, has become significantly slower. The fix often involves removing a slow check or tuning a large thread pool.

6. What strategies can you use to reduce the overall size of a Maven project or its final artifacts?

A: You can reduce size by aggressively using dependency scopes like **provided** (for libraries the server supplies) and **test** (for libraries only needed for testing). I would also use the **<exclusion>** tag to block any unnecessary or duplicate **transitive dependencies** from being included in the final packaged artifact.

Profiles, Environments & Advanced Usage

7. What are Maven profiles, and how can you use them to manage different environments?

A: Maven **profiles** are custom sets of configuration changes defined in the pom.xml that you can activate based on the environment (e.g., dev, prod). You use them to manage different environments by defining environment-specific properties (like database URLs or API keys) that only get applied when that profile is explicitly activated.

8. How do you enforce coding standards and static analysis tools (Checkstyle, PMD, SpotBugs)?

A: I enforce coding standards by integrating these static analysis tools as **plugins** into the verify phase of the Maven lifecycle. I set them up to **fail the build** if they detect violations of the defined coding style (Checkstyle) or potential bugs (SpotBugs/PMD).

9. How can Git or CI tools like Jenkins use Maven in automated pipelines?

A: Git and CI tools use Maven by executing standard commands like **mvn clean install** or **mvn deploy**. In a pipeline, the Git webhook triggers Jenkins to check out the latest code, and Jenkins then runs the Maven commands to compile the code, run tests, and package the final application artifact for deployment.